

Tecnología Digital VI

Optimización, Hiperparámetros y más

Licenciatura en Tecnología Digital - UTDT

Optimización

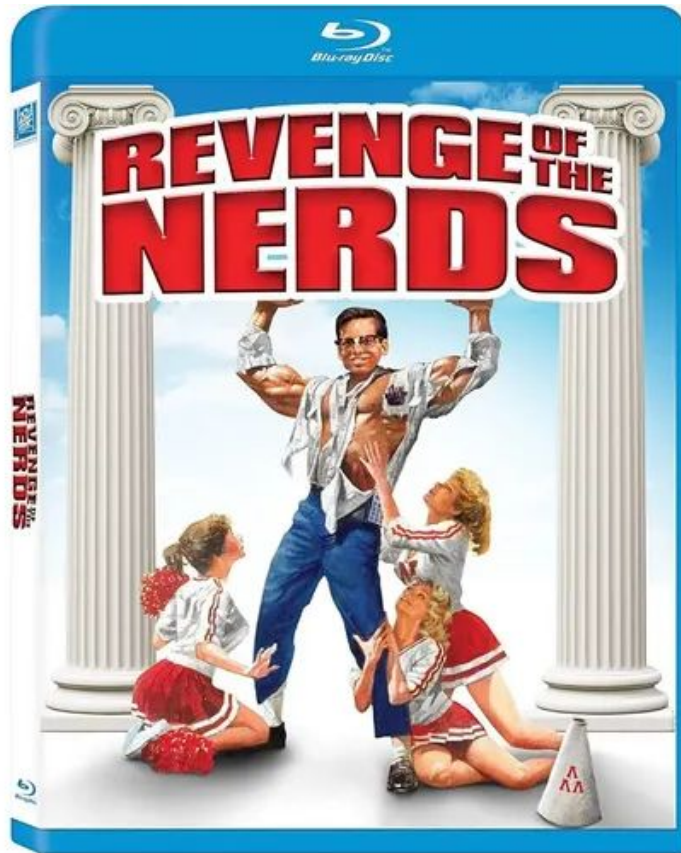
En este punto, tenemos todo lo que define nuestra red: Arquitectura, cantidad de capas, cantidad de neuronas, funciones de activación, *loss function*, cómo usar la red *hacia adelante*, cómo usar la red *hacia atrás*. (Luego seguiremos charlando cómo definimos todo esto en la práctica).

Una vez que está todo esto definido, lo único que nos queda es encontrar los mejores pesos (W) de la red, los que mejor explican al conjunto de entrenamiento *esperando* que el comportamiento se generalice bien al resto de los datos.

Optimización

Es decir, una vez que está todo lo demás definido, el problema se reduce a optimizar (encontrar un mínimo) de la *loss function* en función de los pesos.

TODO lo reducimos a esto: clasificar cosas, predecir un valor numérico, detectar cosas en imágenes, traducir textos, crear textos nuevos, crear imágenes nuevas, etc.



Optimización

Las loss functions de la redes neuronales suelen ser funciones con formas irregulares y complejas. La minimización de estas funciones no se puede hacer de forma análitica (derivar e igualar a 0).

Todos los métodos que se utilizan en la práctica son iterativos. Es decir, comenzamos con la red no entrenada, con sus pesos en valores aleatorios, y luego la vamos entrenando *época tras época* para ir minimizando la loss function.

Tratar de entender la forma de la loss functions es un tema bastante complejo.

Existen desarrollos de visualizaciones para comprender las similitudes y diferencias, buscando explicar porque algunas redes pueden resultar muy fáciles de entrenar mientras que otras son muy complicadas.

Visualizing the Loss Landscape of Neural Nets

Hao Li¹, Zheng Xu¹, Gavin Taylor², Christoph Studer³, Tom Goldstein¹

¹University of Maryland, College Park ²United States Naval Academy ³Cornell University
{hao11,xuzh,tomg}@cs.umd.edu, taylor@usna.edu, studer@cornell.edu

Abstract

Neural network training relies on our ability to find “good” minimizers of highly non-convex loss functions. It is well-known that certain network architecture designs (e.g., skip connections) produce loss functions that train easier, and well-chosen training parameters (batch size, learning rate, optimizer) produce minimizers that generalize better. However, the reasons for these differences, and their effects on the underlying loss landscape, are not well understood. In this paper, we explore the structure of neural loss functions, and the effect of loss landscapes on generalization, using a range of visualization methods. First, we introduce a simple “filter normalization” method that helps us visualize loss function curvature and make meaningful side-by-side comparisons between loss functions. Then, using a variety of visualizations, we explore how network architecture affects the loss landscape, and how training parameters affect the shape of minimizers.

1 Introduction

Training neural networks requires minimizing a high-dimensional non-convex loss function – a task that is hard in theory, but sometimes easy in practice. Despite the NP-hardness of training general neural loss functions [2], simple gradient methods often find global minimizers (parameter configurations with zero or near-zero training loss), even when data and labels are randomized before training [42]. However, this good behavior is not universal; the trainability of neural nets is highly dependent on network architecture design choices, the choice of optimizer, variable initialization, and a variety of other considerations. Unfortunately, the effect of each of these choices on the structure of the underlying loss surface is unclear. Because of the prohibitive cost of loss function evaluations (which requires looping over all the data points in the training set), studies in this field have remained predominantly theoretical.

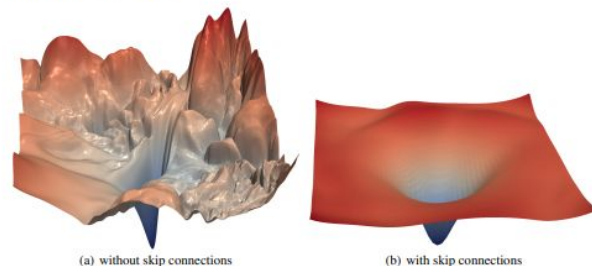


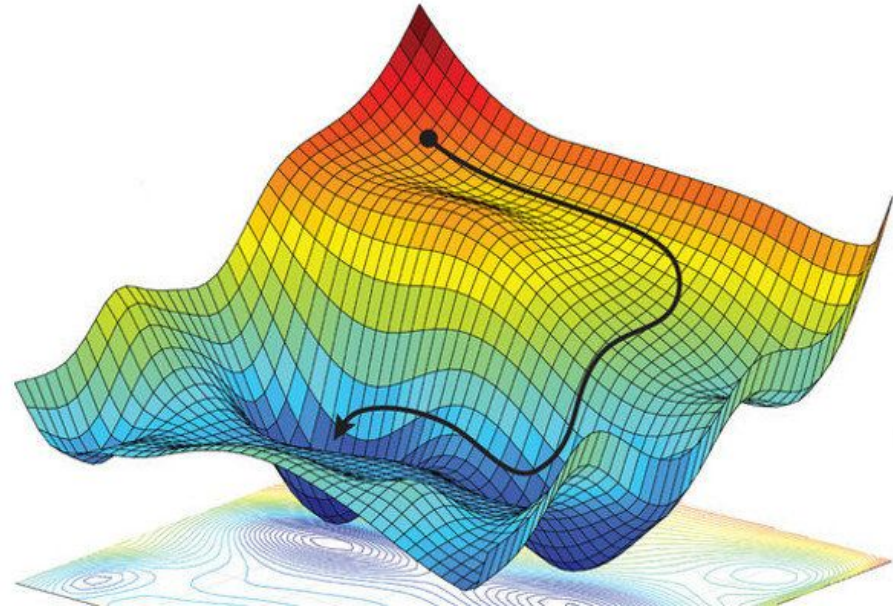
Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures. 32nd Conference on Neural Information Processing Systems (NIPS 2018), Montréal, Canada.

Gradient Descent – Non-Convex

¿Qué pasa si la función no es convexa?

GD podría converger un mínimo local.

No es necesariamente un problema porque igualmente es un mínimo local y puede servir, pero nos podemos perder de explorar otras regiones.

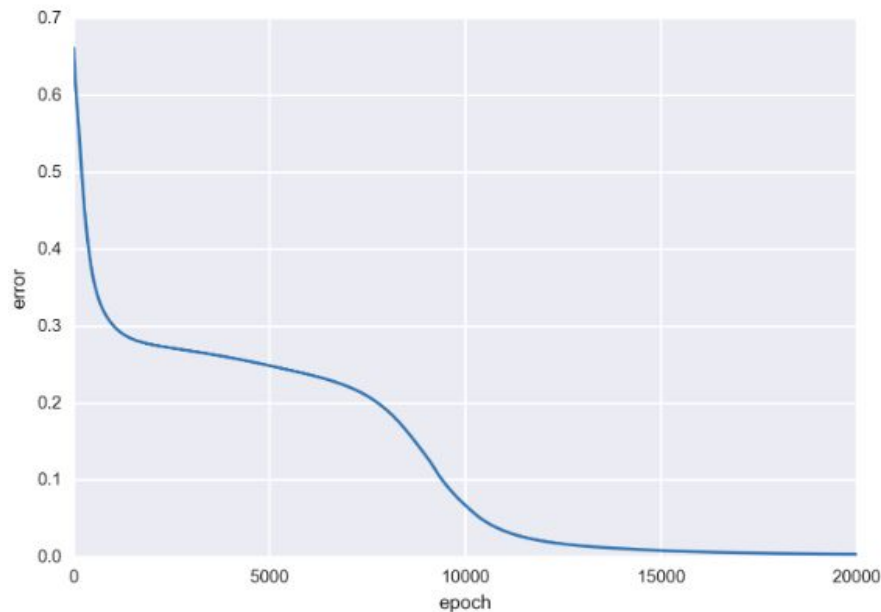


Batch Gradient Descent

Una primera idea posible sería usar todo el dataset de entrenamiento entero para calcular la loss function actual y hacer un paso de GD.

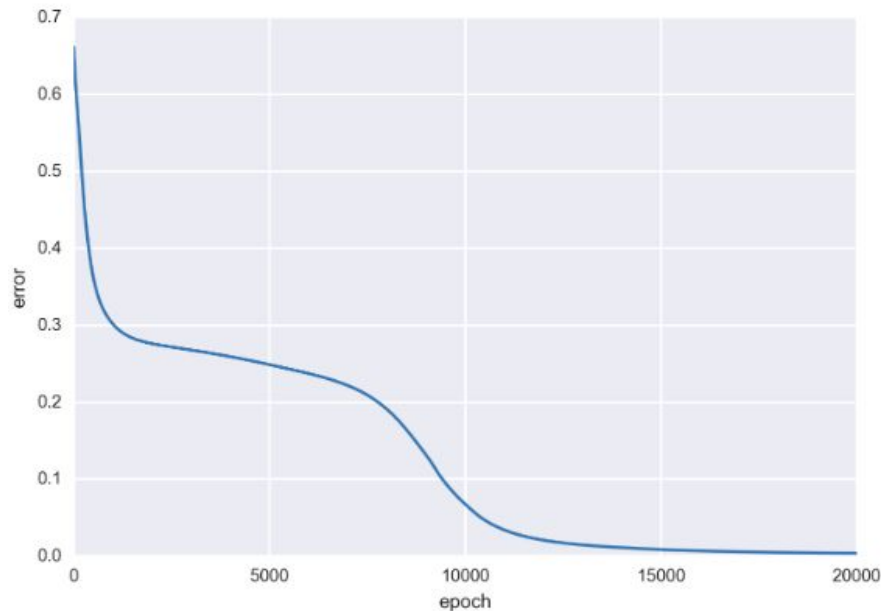
Teóricamente es ideal porque tengo la información completa de lo que pasa con todos mis datos.

Una *época* es pasar todos los datos de entrenamiento por la red y actuar en consecuencia.



Batch Gradient Descent

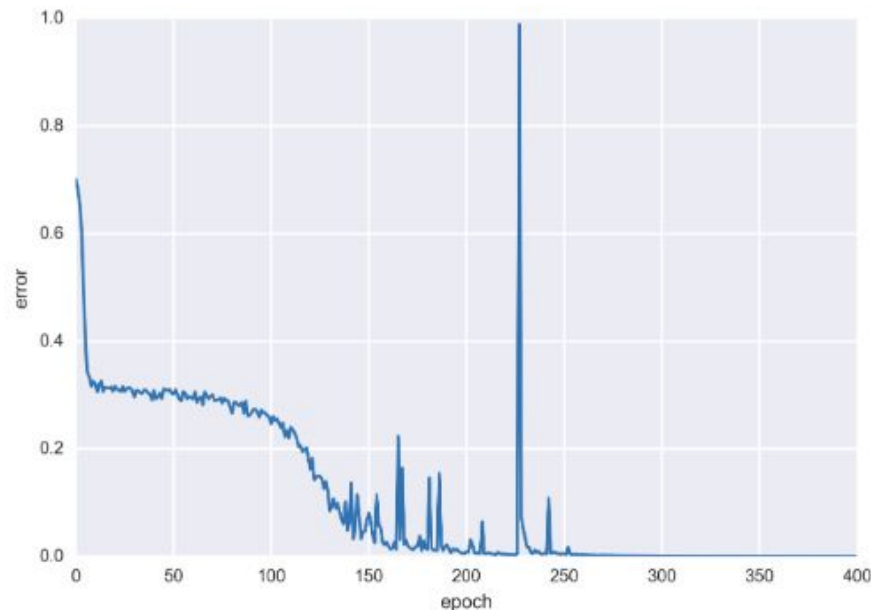
- **Ventajas:**
 - Entrenamiento estable, convergencia a mínimos (al menos locales).
 - No sensible a outliers, curvas suaves.
- **Desventajas:**
 - Impracticable para el Deep Learning moderno (datasets de millones de imágenes, “toda la internet”).



Stochastic Gradient Descent (Puro)

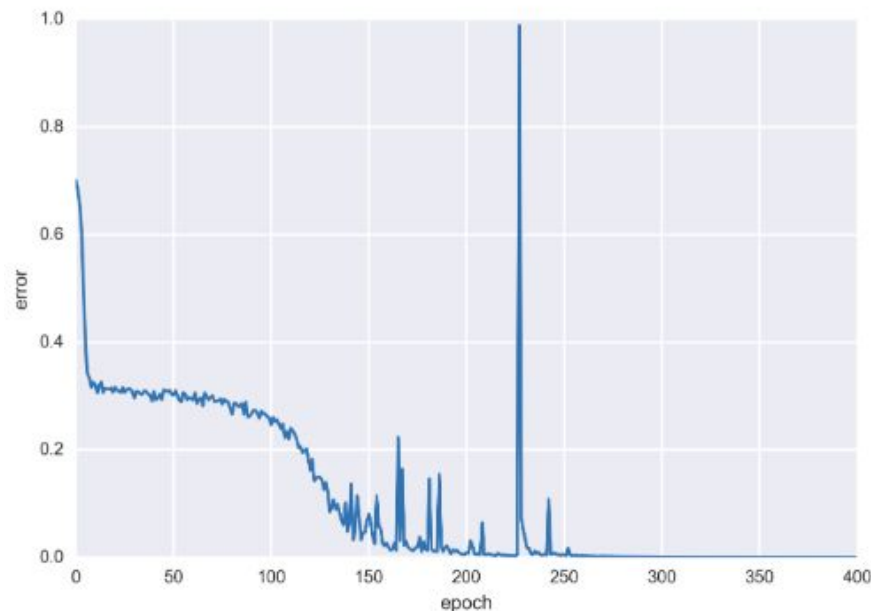
Nos vamos al otro extremo. Si usar todo el dataset es muy costoso, ¿por qué no usamos literalmente una sola muestra?

En su versión pura, lo que hacemos es elegir una sola muestra al azar y actualizar los pesos en función de la loss function para dicha muestra.



Stochastic Gradient Descent (Puro)

- Ventajas:
 - El costo computacional y de memoria es completamente manejable para cualquier problema.
 - Puede converger a buenas soluciones incluso sin ver todos los datos.
- Desventajas:
 - Curva de entrenamiento completamente ruidosa.
 - La trayectoria en el landscape es mucho más caótica (a veces un pro).
 - No se usa en la práctica porque no aprovecha la vectorización.

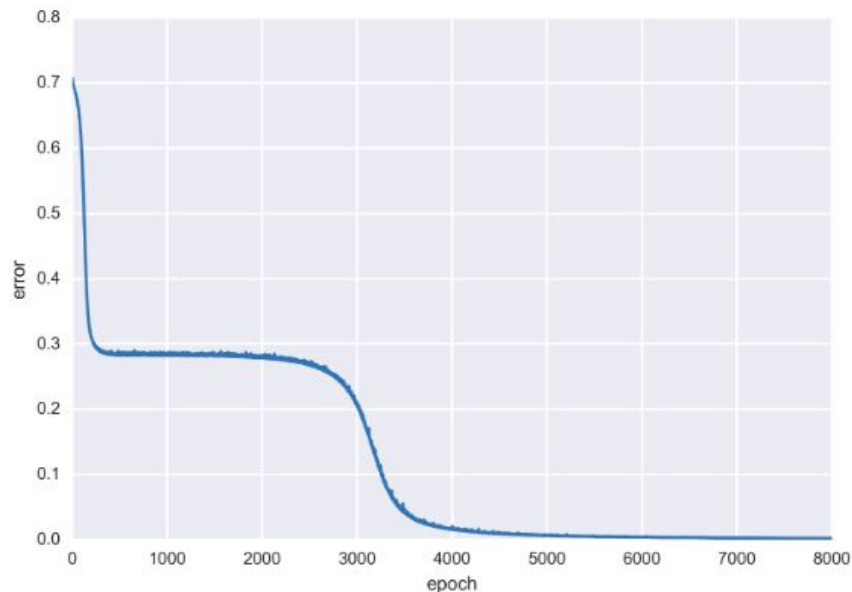


Mini-Batch

El punto intermedio entre los dos extremos. No usamos ni un dato, ni todo el dataset a la vez.

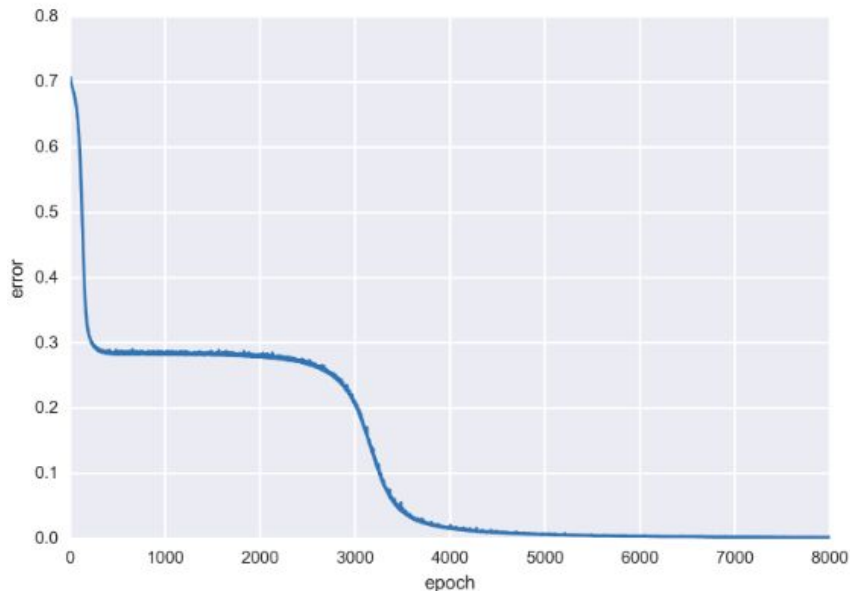
Se usan *mini-batches* de una cierta cantidad de muestras (2, 4, 8, 16, 32, 64). Para cada mini-batch calculamos la loss function y actualizamos en consecuencia.

Una época es pasar todo el dataset dividido en mini-batches por la red.



Mini-Batch

- **Ventajas:**
 - Mucho menos ruidoso que el SGD puro, pero manejable a nivel computacional y en memoria.
 - Explota la vectorización de GPUs.
- **Desventajas:**
 - El tamaño del batch se convierte en un parámetro más a manejar.
 - Batch más grande: más pesado computacionalmente y en memoria, suele ser más preciso.
 - Batch más chico: es más ruidoso pero manejable.



Problemas de GD – SGD

Sea con batches de 1, 64 o n muestras, el algoritmo que estamos utilizando es completamente *memory-less* y *miope*. Solo define cómo actualizar los pesos en función de lo que está pasando en esta época, sin recordar lo que pasó hasta ahora ni prever lo que va a pasar después.

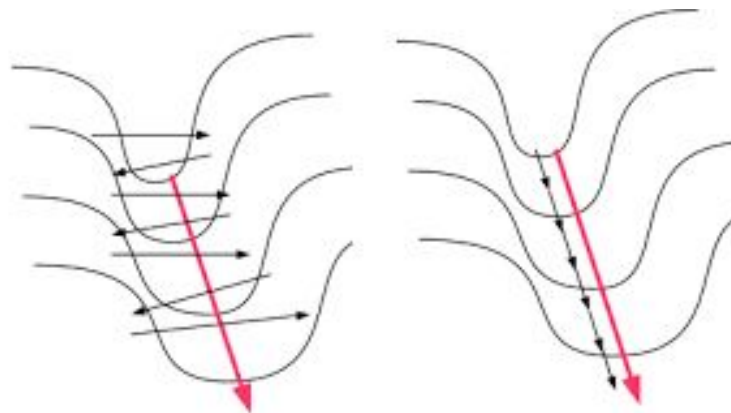
Esto puede traer algunos problemas como:

- *Ravine*: si pensamos en una topología como un *cañón*, la pendiente hacia el centro del cañón puede ser mayor que la pendiente hacia la desembocadura, produciendo un *zig-zag*.
- *Mesetas y puntos silla*: cuando la pendiente es casi cero, el SGD no progresa.

Problemas de GD – SGD

Nos gustaría ir hacia la desembocadura, pero el gradiente nos lleva siempre hacia el centro del cañón.

Si hace 20 pasos que estamos haciendo $+1, -1, +1, -1, \dots, +1, -1$ en la dirección x , tal vez deberíamos darnos cuenta que en realidad no había que moverse en la dirección x .



Momentum

Tanto el problema de *ravine* como el de mesetas puede ser solucionado en varios casos por la noción de *momentum* (o inercia, aunque no es lo mismo).

En vez de calcular el gradiente en la iteración actual como algo aislado e independiente vamos a ir acumulando gradientes de pasos anteriores. En el caso más simple de aplicación de *momentum*, simplemente haremos que la actualización actual sea una combinación entre el gradiente actual y el anterior.

Momentum

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α .

Require: Initial parameter θ , initial velocity v .

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$

 Apply update: $\theta \leftarrow \theta + \mathbf{v}$

end while

- Ravine: Hacía la desembocura, se acumulan. Hacía el centro del cañon, se cancelan.
- Mesetas: No frenamos tanto gracias a los gradientes anteriores.

Learning Rate Adaptativo

El *momentum* es un concepto bastante útil pero en la práctica todavía nos queda el problema de definir un Learning Rate. Es un parámetro que no es simple de definir y que impacta significativamente en el optimizador.

Más allá de la definición en sí misma, no es claro que sea una buena idea mantener un mismo learning rate en toda época y para todo parámetro.

De esta manera, surgieron las primeras ideas de Learning Rate Adaptativo, comenzando por la adaptación para cada parámetro (luego veremos en el tiempo).

AdaGrad

Una de las primeras ideas de Learning Rate Adaptativo la introdujo el algoritmo AdaGrad. En este algoritmo cada parámetro (los w) tiene su propio Learning Rate.

El algoritmo sigue utilizando un Learning Rate global pero luego este se adapta para cada w en la red.

La propuesta es que el Learning Rate se adapte de manera inversamente proporcional a la acumulación de los cuadrados de los gradientes en todas las épocas anteriores para ese parámetro.

Si un parámetro se viene moviendo demasiado, el learning rate baja para buscar convergencia. Si un parámetro no viene moviéndose nada, le permitimos mayor flexibilidad.

Tener en cuenta toda la historia de los gradientes por igual puede no ser lo correcto. Si los primeros movimientos de un parámetro fueron muy grandes (que además puede tener mucho sentido), la cuenta de AdaGrad solo sigue acumulando siempre achicando el escalamiento del Learning Rate.

En la práctica esto puede traducirse a que la red rápidamente se estanca en su entrenamiento.

Geoffrey Hinton en el 2012 propuso RMSProp que agrega un decaimiento exponencial de la acumulación de los gradientes con el objetivo de ir *olvidándose* de los gradientes más viejos.

En la práctica, para redes profundas, RMSProp funcionaba mejor que AdaGrad en la mayoría de los casos.

Tenemos la idea interesante de *momentum*, tenemos la idea interesante de Learning Rate adaptativo con RMSProp. El siguiente paso es mezclar ambas ideas.

Adaptive Moment Estimation (ADAM) es un algoritmo propuesto en 2014 que busca combinar estas ideas. Utiliza la noción de *momentum* para acelerar la convergencia y evitar problemas como el *ravine*, y agrega la noción de Learning Rate Adaptativo para cada parámetro en función de los gradientes anteriores (pesados por su *vejez*).

A día de hoy, sigue siendo uno de los principales algoritmos de optimización para entrenar Redes Neuronales de todo tipo.

Elección de algoritmo

El algoritmo de optimización es aún otra elección posible al momento de desarrollar Redes Neuronales. En líneas generales, se suele utilizar mayormente ADAM, aunque en algunos casos la mezcla de SGD con la noción de *momentum* sigue siendo útil en la práctica.

En general, SGD+Momentum es un poco más ruidoso y *menos preciso* que ADAM, pero para algunas topologías de landscape muy irregulares eso puede llegar a ser algo a favor dado que permite una exploración más libre.

Schedulers

Las adaptaciones al Learning Rate que vimos hasta el momento están localizadas espacialmente, no temporalmente (es decir, la adaptación no depende de la época en la que estemos).

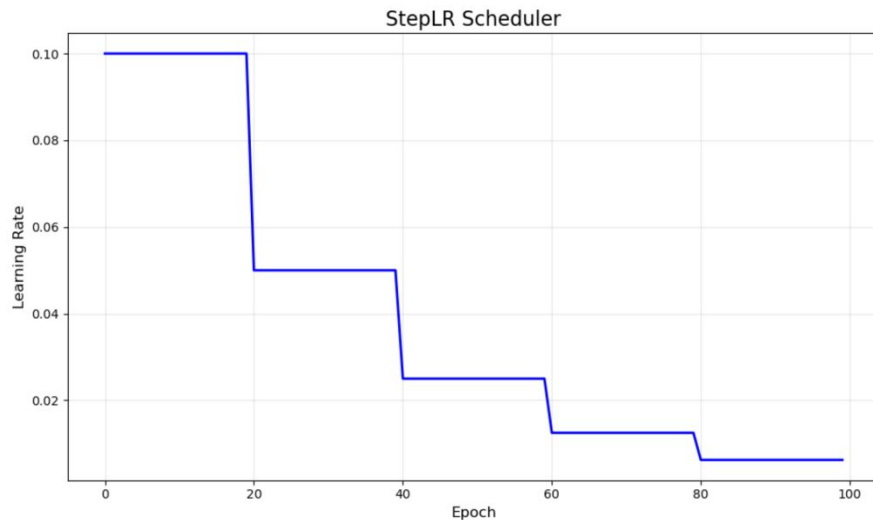
Para adaptar el Learning Rate global en el tiempo se utiliza la noción de *Scheduler*. Es simplemente una función (en función del tiempo) que nos dice cómo se va modificando el Learning Rate global.

Algunos muy utilizados:

- Step decay
- Exponential decay
- Reduce On Plateau
- Cosine Annealing

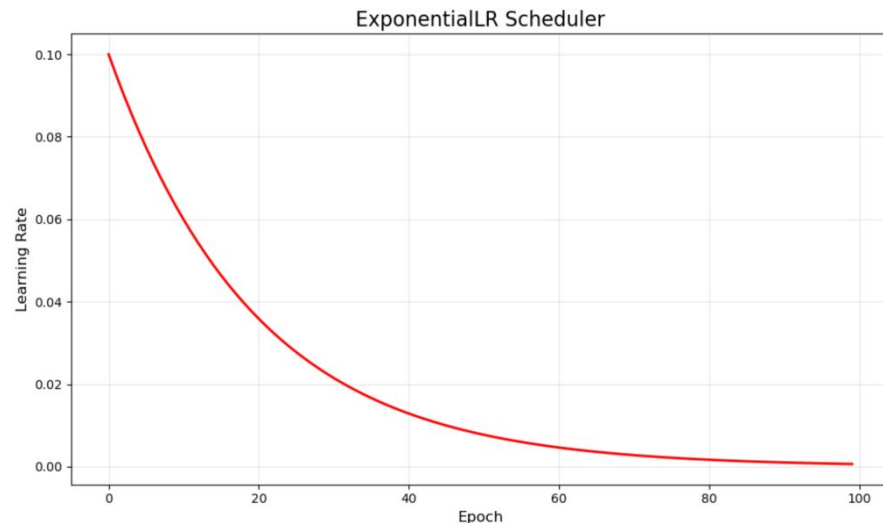
Step decay

Comenzamos con un Learning Rate fijado y cada cierta cantidad de épocas lo dividimos por 2.



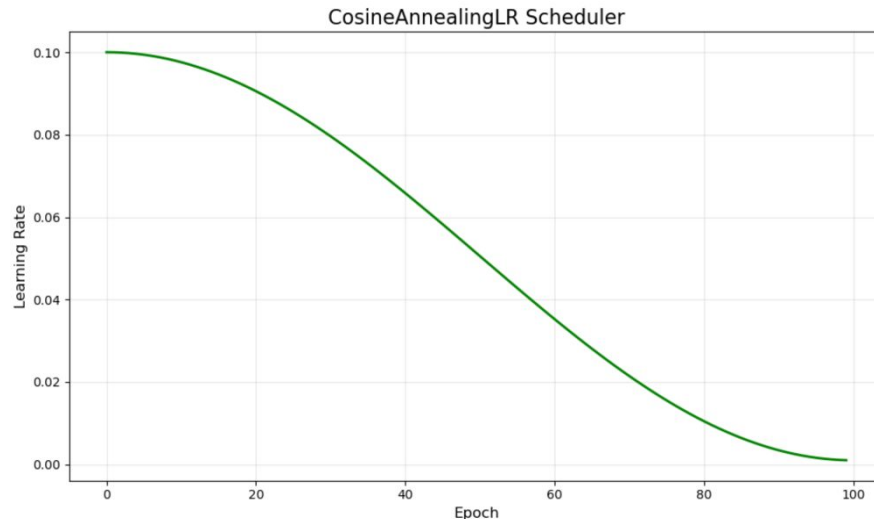
Exponential decay

Comenzamos con un Learning Rate fijado y lo vamos reduciendo constantemente al multiplicarlo por algún escalar menor a 1 (e.g 0.98).



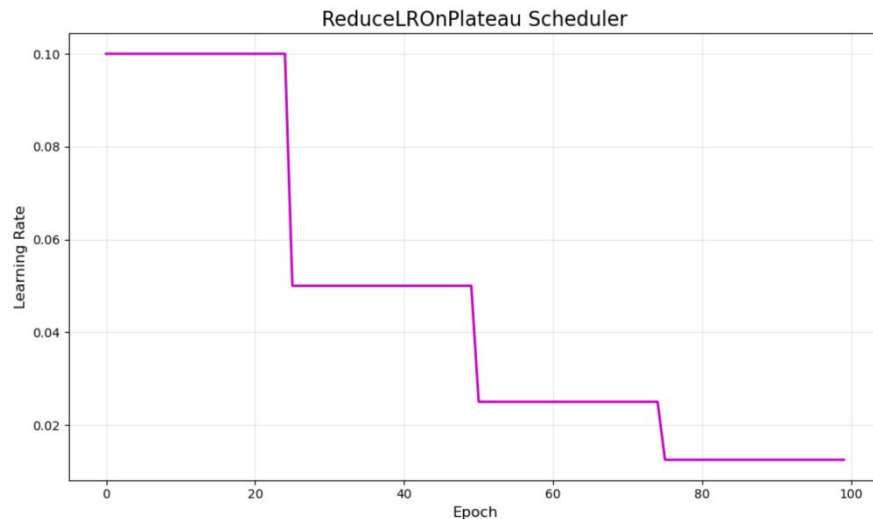
Cosine Annealing

El Learning Rate original se toma como el punto máximo en una curva de coseno y se va actualizando a medida que pasan las épocas.



Reduce on plateau

La reducción del Learning Rate no es fija sino que depende de la evolución del propio entrenamiento. Cuando la red no está mejorando *mucho* su loss function en una cierta cantidad de épocas, se reduce el Learning Rate a la mitad.



Hiperparámetros

Todo lo construido hasta este punto resulta en una gran cantidad de decisiones a tomar antes de poder entrenar nuestro modelo.

Los parámetros son todos los pesos que el modelo va a aprender cuando lo pongamos a entrenar. Los hiperparámetros son los valores que tenemos definir “de antemano” para poder entrenar un modelo.

En una aproximación un poco burda podríamos decir que los hiperparámetros son aquellos valores que define la persona, mientras que los parámetros son aquellos que el modelo aprende por su cuenta.

Hiperparámetros y más

(algunas) Decisiones a tomar antes de entrenar un modelo:

- Learning Rate inicial.
- Arquitectura.
 - Cantidad de capas.
 - Cantidad de neuronas.
 - Funciones de activación.
 - En algunos casos, loss function.
- Batch size.
- Algoritmo de optimización.

No hay una receta ganadora. Hay combinaciones conocidas por su buen desempeño en ciertos tipos de problemas. También hay formas de moverse por el espacio de estas decisiones para buscar la mejor combinación.

Optimización de hiperparámetros

Algunas estrategias conocidas para buscar las mejores combinaciones:

- Random Search.
- Grid Search.
- Optimización Bayesiana.

Con todo lo visto hasta ahora tenemos un panorama amplio de todos los componentes involucrados en la definición y entrenamiento de una Red Neuronal.

Nuestro camino sigue ahora con Redes Neuronales específicamente pensadas para ciertos contextos de aplicación como el tratamiento de imágenes o texto.

Antes de pasar a ellas, vamos a mencionar algunos conceptos *sueltos* con mucha utilidad teórica y práctica en Redes Neuronales.

Input Scaling

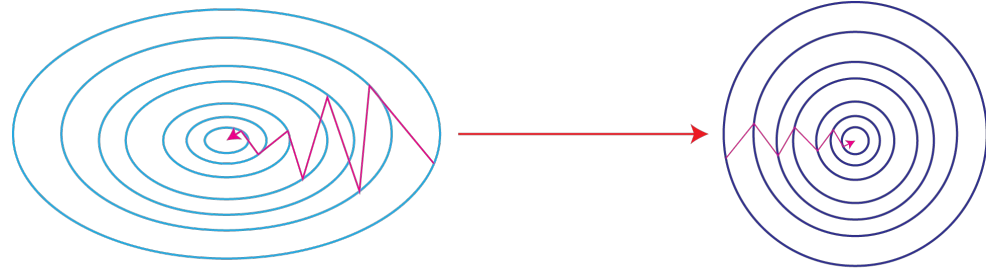
Las redes neuronales son muy sensibles a los rangos de los inputs.

Recordando lo que nos sucedía en Machine Learning clásico, si el modelo opera con nociones de distancia, en general es *necesario* estandarizar los rangos de los diferentes inputs.

Si por un lado tengo el peso de una persona en toneladas, y por el otro lado tengo su altura en milímetros, va ser muy costoso para la red actualizar los pesos de manera correcta para que ambas variables puedan *competir* de manera justa.

Input Scaling

Estandarizar los inputs (o escalarlos con alguna otra técnica como min-max) ayuda a que el landscape de optimización sea más adecuado para que algoritmos como Gradient Descent puedan converger más rápido.



Inicialización de pesos

La convergencia de la red es también una consecuencia directa del punto de partida para nuestro algoritmo.

Existen diversos algoritmos para inicializar los pesos de una red, con el objetivo de acelerar la convergencia y de evitar problemas puntuales como los *vanishing gradients* y *exploding gradients*.

Alternativas:

- Inicialización Random (Uniforme o Normal).
- LeCun initialization.
- Xavier initialization.
- He initialization.

Internal Covariate Shift

Con las técnicas de inicialización de pesos y estandarización del input podemos manejar las distribuciones de los datos de entrada a la primera capa para que tengan formas deseables (e.g. campanas normales centradas en cierto valor).

Esto no previene que las entradas a la capas ocultas o a la capa de output tengan corrimientos en su distribución a medida que la red se va entrenando que pueden derivar en diferentes problemas.

Batch Normalization

Existen capas adicionales en las Redes Neuronales que se agregan con el único objetivo de corregir ciertos problemas como la regularización o el corrimiento de las distribuciones de los datos.

Batch Normalization es una de estas capas más utilizadas. Su función es normalizar los valores de cada batch que se está utilizando para entrenar la red antes de ingresar a ciertas capas (es decir se coloca justo antes de una capa oculta).

Existen otros tipos de normalizaciones muy utilizadas *dentro* de la red como LayerNorm.

Al igual que todo otro modelo de Machine Learning, las Redes Neuronales se someten a diferentes técnicas de regularización para evitar que la red sobreajuste a los datos de entrenamiento y luego generalice con peor performance.

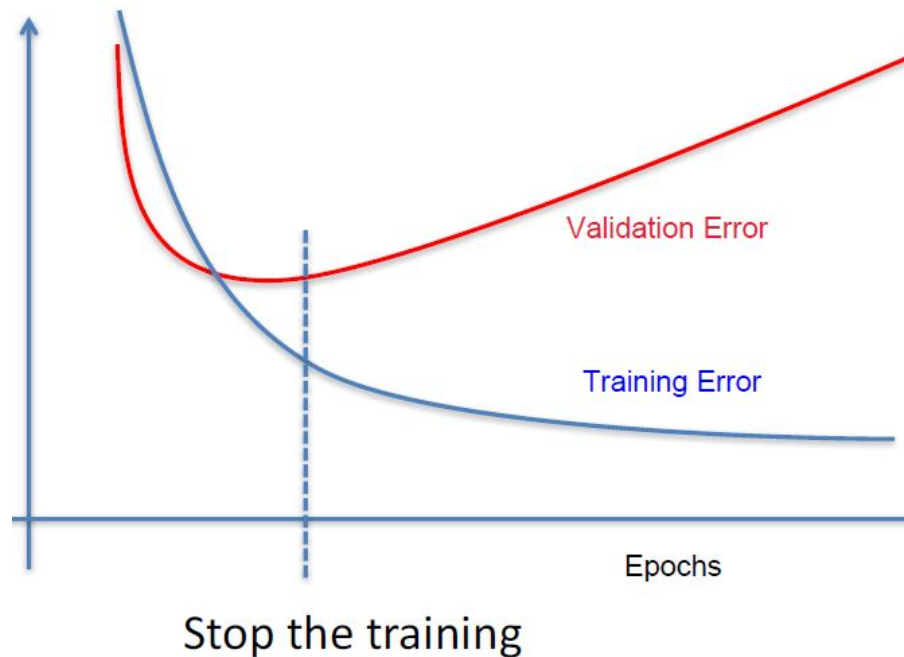
Algunas técnicas de regularización en Redes Neuronales:

- Early Stopping.
- L1 y L2.
- Dropout.

Early Stopping

Al entrenar siempre estamos mirando la loss function en el conjunto de train, pero nada impide al terminar cada época usar la red en modo inferencia para calcular la loss function en el conjunto de validación.

Una técnica simple pero efectiva es cortar el entrenamiento cuando los errores de entrenamiento y validación comienza a diverger (o guardar la red en cada época para luego volver a este punto).



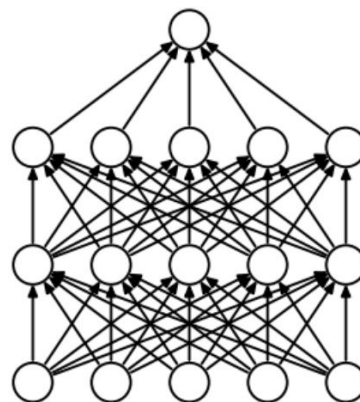
Al igual que sucedió en Regresión Logística, podemos agregar *funciones de presupuesto* en la loss function que busquen restringir los valores que pueden tomar los pesos de la red.

Lo que se hace es sumar este tipo de funciones (como Lasso o Ridge) en la loss function y luego simplemente seguir con el entrenamiento habitual.

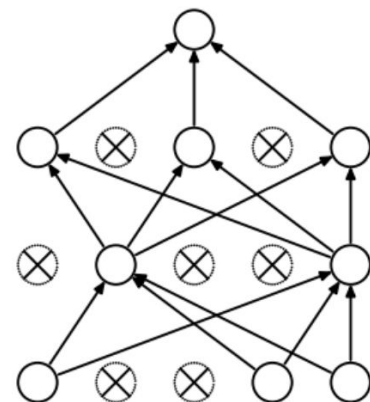
Dropout

La técnica de *dropout* lo que hace es, cada vez que un mini-batch pasa por la red, apagar al azar una cierta cantidad de neuronas.

Evita que ciertos patrones en el dataset de entrenamiento sean aprendidos “de memoria” por ciertas neuronas al obligar la búsqueda de aprendizajes más robustos



(a) Standard Neural Net



(b) After applying dropout.

Dropout

En la práctica el *dropout* se pone en cada capa por separado.

En PyTorch se agrega como una capa más pero que en realidad no tiene ningún parámetro a aprender.

Es importante notar que estas capas solo tienen inferencia en el entrenamiento. En tiempo de inferencia usamos todas las neuronas.

```
import torch.nn as nn

modelo = nn.Sequential(
    nn.Linear(100, 50),
    nn.ReLU(),
    nn.Dropout(p=0.3),
    nn.Linear(50, 10)
)

modelo.train()

modelo.eval()
```


Monitoreo de entrenamiento: gradientes

Al entrenar las redes neuronales hay un montón de cuestiones que se pueden ir monitoreando durante el entrenamiento. La foto final es simplemente nuestro modelo completamente entrenado pero lo que sucede en el medio puede ser muy útil.

Los problemas como *vanishing* y *exploding gradients* pueden ser monitoreados en cada momento.

